

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – Sorting



NAMA : Muhammad Ervan Febriano

NIM : 109082500079

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Sorting adalah proses mengurutkan data agar lebih rapi dan mudah diolah. Pada program ini digunakan algoritma Insertion Sort untuk mengurutkan data buku berdasarkan rating dari terbesar ke terkecil. Cara kerja insertion sort yaitu mengambil satu data lalu membandingkannya dengan data sebelumnya sampai posisi yang sesuai ditemukan. Proses ini dilakukan terus sampai semua data terurut.

Selain sorting, program juga menggunakan Binary Search untuk mencari buku berdasarkan rating tertentu. Binary search bekerja dengan mengecek data dari bagian tengah sehingga proses pencarian menjadi lebih cepat dibanding pencarian biasa. Namun metode ini hanya bisa digunakan jika data sudah dalam keadaan terurut terlebih dahulu.

B. Guided

1. [guided1.go]

```
package main

import "fmt"

// Fungsi Selection Sort untuk mengurutkan array dari kecil ke besar
func selectionSortArray(angka *[5]int) {
    var idxMin, i, j int

    // Loop luar untuk iterasi sorting
    for i = 0; i < len(angka)-1; i++ {
        idxMin = i

        // Loop dalam untuk mencari nilai terkecil
        for j = i + 1; j < len(angka); j++ {
            if angka[j] <= angka[idxMin] {
                idxMin = j
            }
        }

        // Tukar jika ditemukan nilai lebih kecil
        if idxMin != i {
            angka[i], angka[idxMin] = angka[idxMin], angka[i]
        }
    }
}

func main() {
    var arrAngka [5]int
```

```

// Input data
for i := 0; i < len(arrAngka); i++ {
    fmt.Printf("Masukkan data angka ke-%d : ", i)
    fmt.Scan(&arrAngka[i])
}

fmt.Println()
fmt.Println("=== SEBELUM DISORTING ===")
for i := 0; i < len(arrAngka); i++ {
    fmt.Print(arrAngka[i], " - ")
}

// Proses sorting
selectionSortArray(&arrAngka)

fmt.Println("\n=== SETELAH DISORTING ===")
for i := 0; i < len(arrAngka); i++ {
    fmt.Print(arrAngka[i], " - ")
}
}

```

Output :

```

PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_
\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
Masukkan data angka ke-0 : 5
Masukkan data angka ke-1 : 7
Masukkan data angka ke-2 : 3
Masukkan data angka ke-3 : 9
Masukkan data angka ke-4 : 1

=== SEBELUM DISORTING ===
5 - 7 - 3 - 9 - 1 -
=== SETELAH DISORTING ===
1 - 3 - 5 - 7 - 9 -

```

2. [guided2.go]

```

package main

import "fmt"

type mahasiswa struct {

```

```

    nama string
    nim  string
    IPK  float64
}

// Fungsi Selection Sort berdasarkan IPK (kecil ke besar)
func selectionSortArray(sMHS *[5]mahasiswa) {
    var idxMin, i, j int

    // Loop luar untuk iterasi sorting
    for i = 0; i < len(sMHS)-1; i++ {
        idxMin = i

        // Loop dalam untuk mencari IPK terkecil
        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idxMin].IPK {
                idxMin = j
            }
        }

        // Tukar data jika ditemukan IPK lebih kecil
        if idxMin != i {
            sMHS[i], sMHS[idxMin] = sMHS[idxMin], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    // Input data mahasiswa
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)

        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)

        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }

    // Menampilkan data sebelum sorting
    fmt.Println()

```

```

fmt.Println("=== SEBELUM SORTING ===")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM  : %s\n", structMHS[i].nim)
    fmt.Printf("IPK   : %.2f\n", structMHS[i].IPK)
}
fmt.Println("=====")
fmt.Println()

// Proses sorting
selectionSortArray(&structMHS)

// Menampilkan data setelah sorting
fmt.Println("=== SETELAH SORTING ===")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM  : %s\n", structMHS[i].nim)
    fmt.Printf("IPK   : %.2f\n", structMHS[i].IPK)
}
fmt.Println("=====")
fmt.Println()
}

```

Output :

```

PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\10908250079_Muhammad_Ervan_IKUM\10908250079_Muhammad_Ervan_Febriano\10908250079_Muhammad_Ervan_Febriano\

=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : Ervan
NIM : 1090
IPK : 3.50
--- Data Mahasiswa Ke-1 ---
Nama : rafi
NIM : 1091
IPK : 2.00
--- Data Mahasiswa Ke-2 ---
Nama : yajid
NIM : 1092
IPK : 3.00
--- Data Mahasiswa Ke-3 ---
Nama : faiq
NIM : 1093
IPK : 2.40
--- Data Mahasiswa Ke-4 ---
Nama : hanip
NIM : 3.2
IPK : 3.00
=====

=== SETELAH SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : rafi
NIM : 1091
IPK : 2.00
--- Data Mahasiswa Ke-1 ---
Nama : faiq
NIM : 1093
IPK : 2.40
--- Data Mahasiswa Ke-2 ---
Nama : yajid
NIM : 1092
IPK : 3.00
--- Data Mahasiswa Ke-3 ---
Nama : hanip
NIM : 3.2
IPK : 3.00
--- Data Mahasiswa Ke-4 ---
Nama : Ervan
NIM : 1090
IPK : 3.50
=====

```

3. [guided3.go]

```

package main

import "fmt"

```

```

func selecetionSortArray(angka *[5]int){
    var idx_min, i, j int
    for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari nilai
ekstrim)
            if angka[j] <= angka[idx_min] { //sorting terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            angka[i], angka[idx_min] = angka[idx_min], angka[i]
        }
    }
}

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selecetionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

Output:

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\10908250079_Muhammad_Ervan_Febriano\Documents\ENGGINER\Alpro\PRAKTIKUM\10908250079_Muhammad_Ervan_Febriano\go"
5
12 4 9 25 1
25 12 9 4 1
```

4. [guided4.go]

Output:

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\10908250079_Muhammad_Ervan_Febriano\Documents\ENGGINER\Alpro\PRAKTIKUM\10908250079_Muhammad_Ervan_Febriano\tring.go"
4
pisang apel mangga jeruk
apel jeruk mangga pisang
```

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

Jawaban :

```
package main
import "fmt"

const NMAX = 1000
type ArrRumah [NMAX]int

func selectionSort(A *ArrRumah, n int) {
    var i, j, idxMin, temp int
```



```

    for i = 0; i < n-1; i++ {
        idxMin = i

        for j = i + 1; j < n; j++ {
            if A[j] < A[idxMin] {
                idxMin = j
            }
        }

        temp = A[i]
        A[i] = A[idxMin]
        A[idxMin] = temp
    }
}

func main() {
    var n, m int
    var rumah ArrRumah

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSort(&rumah, m)

        for j := 0; j < m; j++ {
            fmt.Print(rumah[j], " ")
        }
        fmt.Println()
    }
}

```

Output :

```
PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\10908250079_Muhammad
\Documents\ENGINER\Alpro\PRAKTIKUM\10908250079_Muhammad_Ervan_Febriano\
go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
```

Penjelasan :

[Program Go ini dipakai buat mengurutkan angka dari yang paling kecil ke paling besar memakai algoritma selection sort. Jadi pertama user masukan jumlah data (n), lalu setiap data punya beberapa angka (m) yang disimpan di array rumah. Setelah semua angka dimasukan, function selectionSort bakal nyari angka terkecil satu per satu lalu ditukar posisinya ke depan sampai semua data terurut. Di dalam selection sort ada perulangan buat nyari indeks angka paling kecil (idxMin), terus ditukar pakai variabel sementara temp. Setelah proses sorting selesai, hasil array yang sudah rapi langsung ditampilkan ke layar. Intinya program ini latihan dasar array, perulangan, function, dan algoritma sorting di bahasa Go.]

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil. Format

Masukan masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

Jawaban :

```
package main
import "fmt"
```

```

const NMAX = 1000
type ArrRumah [NMAX]int

func selectionAsc(A *ArrRumah, n int) {
    var i, j, idxMin, temp int

    for i = 0; i < n-1; i++ {
        idxMin = i
        for j = i + 1; j < n; j++ {
            if A[j] < A[idxMin] {
                idxMin = j
            }
        }
        temp = A[i]
        A[i] = A[idxMin]
        A[idxMin] = temp
    }
}

func selectionDesc(A *ArrRumah, n int) {
    var i, j, idxMax, temp int

    for i = 0; i < n-1; i++ {
        idxMax = i
        for j = i + 1; j < n; j++ {
            if A[j] > A[idxMax] {
                idxMax = j
            }
        }
        temp = A[i]
        A[i] = A[idxMax]
        A[idxMax] = temp
    }
}

func main() {
    var n, m, x int
    var ganjil, genap ArrRumah
    var ng, ne int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        ng = 0
    }
}

```

```

        ne = 0

        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&x)

            if x%2 != 0 {
                ganjil[ng] = x
                ng++
            } else {
                genap[ne] = x
                ne++
            }
        }

        selectionAsc(&ganjil, ng)
        selectionDesc(&genap, ne)

        for j := 0; j < ng; j++ {
            fmt.Print(ganjil[j], " ")
        }

        for j := 0; j < ne; j++ {
            fmt.Print(genap[j], " ")
        }

        fmt.Println()
    }
}

```

Output :

```

● PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad
  \Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
  go"
  3
  5 2 1 7 9 13
  1 7 9 13 2
  6 189 15 27 39 75 133
  15 27 39 75 133 189
  3 4 9 1
  1 9 4

```

Penjelasan :

[Program Go ini berfungsi untuk memisahkan bilangan ganjil dan genap, lalu mengurutkannya dengan cara yang berbeda. Pertama user masukan jumlah kasus (n), lalu setiap kasus punya beberapa angka (m). Saat angka dimasukan, program ngecek apakah angka itu ganjil atau genap memakai $x \% 2$. Kalau ganjil disimpan ke array ganjil, kalau genap masuk ke array genap. Setelah itu array ganjil diurutkan naik (ascending) memakai function selectionAsc, sedangkan array genap diurutkan turun (descending) memakai function selectionDesc. Algoritma yang dipakai tetap selection sort, bedanya cuma tanda perbandingannya aja. Terakhir semua angka ganjil yang sudah urut ditampilkan dulu, baru angka genap yang sudah urut menurun. Jadi inti program ini adalah latihan array, percabangan, perulangan, serta sorting ascending dan descending di bahasa Go.]

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Jawaban:

```
package main

import "fmt"

func insertionSort(arr []int, n int) {
    var i, j, key int
    for i = 1; i < n; i++ {
        key = arr[i]
        j = i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}
```

```

    }
}

func cekJarak(arr []int, n int) {
    if n <= 1 {
        fmt.Println("Data berjarak 0")
        return
    }

    selisih := arr[1] - arr[0]

    for i := 2; i < n; i++ {
        if arr[i]-arr[i-1] != selisih {
            fmt.Println("Data berjarak tidak tetap")
            return
        }
    }

    fmt.Printf("Data berjarak %d\n", selisih)
}

func main() {
    var arr [100]int
    var x, n int

    for {
        fmt.Scan(&x)
        if x < 0 {
            break
        }
        arr[n] = x
        n++
    }

    insertionSort(arr[:], n)

    for i := 0; i < n; i++ {
        fmt.Print(arr[i], " ")
    }
    fmt.Println()
    cekJarak(arr[:], n)
}

```

Output :

```
● PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad
  \Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
  go"
  31 13 25 43 1 7 19 37 -5
  1 7 13 19 25 31 37 43
  Data berjarak 6
● PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad
  \Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
  go"
  4 40 14 8 26 1 38 2 32 -31
  1 2 4 8 14 26 32 38 40
  Data berjarak tidak tetap
```

Penjelasan :

[Program Go ini digunakan untuk menerima beberapa angka dari user lalu mengurutkannya memakai algoritma insertion sort. Input akan terus dimasukkan sampai user mengetik angka negatif, karena angka negatif dipakai sebagai tanda berhenti input. Setelah semua data masuk, function insertionSort bekerja dengan cara mengambil satu angka (key) lalu membandingkannya dengan angka sebelumnya sampai posisi yang benar ditemukan, jadi hasil akhirnya urut dari kecil ke besar. Setelah data ditampilkan, program lanjut mengecek apakah jarak antar angka selalu sama memakai function cekJarak. Program menghitung selisih dua angka pertama, lalu membandingkannya dengan selisih angka berikutnya. Kalau semua selisih sama maka output menampilkan “Data berjarak x”, tetapi kalau ada yang beda maka muncul “Data berjarak tidak tetap”. Intinya program ini melatih array, looping, sorting insertion sort, dan pengecekan pola selisih antar data.]

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris

terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

```

package main

import "fmt"

const NMAX = 100

type Buku struct {
    judul      string
    penulis    string
    penerbit   string
    tahun      int
    eksemplar  int
    rating     int
}

type DaftarBuku [NMAX]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&pustaka[i].judul,
                &pustaka[i].penulis,
                &pustaka[i].penerbit,
                &pustaka[i].tahun,
                &pustaka[i].eksemplar,
                &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    max := pustaka[0]

    for i := 1; i < n; i++ {
        if pustaka[i].rating > max.rating {
            max = pustaka[i]
        }
    }

    fmt.Println(max.judul, max.penulis, max.penerbit, max.tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    var temp Buku
    var j int

    for i := 1; i < n; i++ {

```



```

        temp = pustaka[i]
        j = i - 1

        for j >= 0 && pustaka[j].rating < temp.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }

        pustaka[j+1] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    batas := 5
    if n < 5 {
        batas = n
    }

    for i := 0; i < batas; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n, r int) {
    kiri := 0
    kanan := n - 1
    var tengah int

    for kiri <= kanan {
        tengah = (kiri + kanan) / 2

        if pustaka[tengah].rating == r {
            fmt.Println(
                pustaka[tengah].judul,
                pustaka[tengah].penulis,
                pustaka[tengah].penerbit,
                pustaka[tengah].tahun,
                pustaka[tengah].eksemplar,
                pustaka[tengah].rating)
            return
        } else if r > pustaka[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }
}

```

```

    }

    fmt.Println("Tidak ada buku dengan rating seperti itu")
}

func main() {
    var pustaka DaftarBuku
    var n, r int

    fmt.Scan(&n)

    DaftarkanBuku(&pustaka, n)
    CetakTerfavorit(pustaka, n)

    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)

    fmt.Scan(&r)
    CariBuku(pustaka, n, r)
}

```

Output :

```

● PS C:\Users\USER\Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad
  \Documents\ENGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
  go"
  6
  LaskarPelangi AndreaBentata Bentang 2005 10 9
  Bumi TereLiye Gramedia 2014 15 8
  Negeri5Menara AhmadFuadi Gramedia 2009 12 7
  Dilan PidiBaiq PastelBooks 2014 20 8
  AyatAyatCinta Habiburrahman Republika 2004 8 10
  CantikItuLuka EkaKurniawan Gramedia 2002 5 9
  AyatAyatCinta Habiburrahman Republika 2004
  AyatAyatCinta
  LaskarPelangi
  CantikItuLuka
  Bumi
  Dilan
  8
  Dilan PidiBaiq PastelBooks 2014 20 8

```

Penjelasan :

[Program Go ini dibuat untuk mengelola data buku dalam sebuah perpustakaan sederhana. Data buku disimpan memakai struct Buku yang berisi judul, penulis, penerbit, tahun, jumlah eksemplar, dan rating. Pertama user memasukkan

jumlah buku (n), lalu function `DaftarkanBuku` digunakan untuk menginput semua data buku ke dalam array `DaftarBuku`. Setelah itu function `CetakTerfavorit` mencari buku dengan rating paling tinggi lalu menampilkan informasi bukunya. Selanjutnya data buku diurutkan berdasarkan rating tertinggi ke terendah memakai algoritma insertion sort pada function `UrutBuku`. Setelah terurut, function `Cetak5Terbaru` menampilkan maksimal 5 judul buku dengan rating terbaik. Terakhir user memasukkan rating yang ingin dicari, lalu function `CariBuku` memakai binary search untuk mencari buku dengan rating tersebut secara lebih cepat. Jika ditemukan maka semua data buku ditampilkan, kalau tidak ada maka muncul pesan bahwa buku tidak ditemukan. Intinya program ini melatih penggunaan struct, array, sorting insertion sort, dan searching binary search di bahasa Go.]

D. Kesimpulan

Kesimpulan dari praktikum ini adalah mahasiswa dapat memahami dasar pemrograman Go melalui penggunaan array, struct, function, perulangan, dan percabangan dalam menyelesaikan berbagai masalah. Selain itu praktikum ini juga melatih penerapan algoritma sorting seperti selection sort dan insertion sort untuk mengurutkan data, serta algoritma searching seperti binary search untuk mencari data dengan lebih efisien. Dari beberapa program yang dibuat, dapat dipahami bahwa pengolahan data dapat dilakukan dengan lebih terstruktur dan rapi menggunakan algoritma yang sesuai. Praktikum ini juga membantu meningkatkan pemahaman logika pemrograman dalam mengolah input, memproses data, dan menampilkan output sesuai kebutuhan.